

Taller 2: Simulación de Despliegue DevOps — La Pared de la Confusión y Automatización

Resultados de Aprendizaje: RAA1, RAA2, RAA3

Facultad de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI, Cali, Colombia

Resumen

Resumen— Este taller presenta una actividad de simulación práctica basada en software para experimentar la transición cultural y tecnológica desde los silos organizacionales tradicionales hacia las prácticas ágiles de DevOps. Los estudiantes se dividirán en equipos y asumirán roles estrictos de Desarrollo (Dev) y Operaciones (Ops) bajo restricciones físicas y de red. A través de dos fases diferenciadas, se medirán métricas clave de desempeño (Lead Time, Deployment Frequency y Change Failure Rate) para contrastar el impacto de la desconexión manual frente a la automatización de infraestructura como código (IaC), configuraciones de cortafuegos (Firewalls) y pipelines integrados.

1. Introducción

El despliegue de software en entornos corporativos tradicionales está frecuentemente plagado de fricciones debido a la desconexión entre los equipos que construyen la aplicación y aquellos que la operan en entornos de producción. Esta desconexión es conocida como la “Pared de la Confusión”, donde los incentivos opuestos (innovación rápida vs. estabilidad operativa) retrasan significativamente el **time-to-market** y aumentan los fallos de configuración.

Esta actividad emula de forma digital la clásica dinámica de simulación con legos (sesión 2), pero utilizando código de software real, direccionamiento IP en red local y políticas activas de seguridad en el firewall.

2. Objetivos de Aprendizaje

Al finalizar este taller, el estudiante estará en capacidad de:

- **Experimentar** la fricción operativa y de comunicación provocada por los silos organizacionales tradicionales.
- **Configurar y depurar** políticas de acceso a nivel de red (Firewalls) para interconexión de servicios distribuidos.
- **Medir y analizar** el impacto de DevOps utilizando métricas ágiles de entrega de software.

3. Diseño y Reglas de la Simulación

Los estudiantes trabajarán en equipos de **5 a 7 integrantes**, divididos internamente en dos subgrupos con reglas y entornos físicos estrictos:

- **Desarrolladores (Devs) [3–4 integrantes]:**
 - Programan en sus laptops personales.
 - No tienen permitido mirar las pantallas ni interactuar físicamente con los servidores de producción gestionados por Ops.
 - Son responsables de codificar las Historias de Usuario (HUs) solicitadas por el negocio.
- **Operaciones (Ops) [2–3 integrantes]:**
 - Administran los computadores físicos asignados como servidores de producción (1 PC por componente).
 - Son responsables de instalar dependencias, configurar el entorno y garantizar el uptime del sistema.
 - Deben tener el firewall de sus sistemas operativos activo (ej. `ufw` en Linux o Windows Firewall) bloqueando todo el tráfico entrante de red, excepto puertos explícitamente autorizados.

3.1. Regla de Comunicación de Silos (Fase 1)

Durante la primera fase del taller, la comunicación directa entre Devs y Ops está ****estrictamente prohibida****. Los desarrolladores deben empaquetar su código (ej. archivos comprimidos `.zip`) y enviarlo a través de un canal asincrónico (ej. correo electrónico o carpeta compartida) adjuntando un archivo de texto con las instrucciones manuales de despliegue. Si hay fallos en producción, Ops solo puede reportarlos de vuelta mediante “tickets escritos” describiendo el síntoma.

3.2. Restricción de Red y Firewall

Los servidores de producción controlados por Ops se dividen en dos componentes físicos:

1. **Servidor Backend (PC 1):** Expone los servicios de cálculo. El cortafuegos de este nodo debe estar activo y bloqueando todas las conexiones entrantes de red, excepto un puerto concreto (ej: puerto 5000 o 8080) que Ops deberá habilitar.
2. **Servidor Frontend (PC 2):** Servidor que interactúa con el cliente (por consola o interfaz web) y consume los endpoints del Backend.

4. Las Historias de Usuario (HUs)

El negocio requiere una calculadora distribuida muy simple escrita en el lenguaje de elección del equipo (ej. Python, Node.js o Java). Cada HU debe completarse en orden secuencial:

HU1: Servicio de Suma (Backend y Frontend)

Como usuario final, quiero ingresar dos números en el Frontend y obtener la suma procesada por el Backend.

- **Criterio de aceptación:** El Frontend realiza una petición HTTP/REST al Backend. El Backend realiza el cálculo y retorna un JSON con el resultado.

HU2: Multi-Operación (Resta y Multiplicación)

Como usuario final, quiero poder restar y multiplicar dos números en la aplicación.

- **Criterio de aceptación:** El Backend incorpora endpoints específicos para resta y multiplicación, y el Frontend permite seleccionarlos.

HU3: Historial del Sistema de Registro (SoR)

Como administrador de la aplicación, quiero que cada cálculo exitoso se registre de forma persistente.

- **Criterio de aceptación:** Cada operación procesada en el Backend debe guardarse en un archivo local en el servidor (simulando una base de datos) y el Frontend debe poder consultar las últimas 5 operaciones registradas.

HU4: Operación de División con Validación (Fase 2)

Como usuario final, quiero poder dividir dos números en la aplicación.

- **Criterio de aceptación:** Endpoint `/divide` en Backend. Si el denominador es 0, debe retornar un error HTTP 400 (`Bad Request`) con un JSON aclarativo y loguearlo. El Frontend debe capturar el error y mostrarlo amigablemente.

HU5: Telemetría de Operaciones / Health Check (Fase 2)

Como administrador, quiero conocer el estado de salud y operatividad de la aplicación.

- **Criterio de aceptación:** Endpoint `/health` en Backend y `/status` en Frontend que retornen un JSON indicando el estado del servicio, el tiempo de actividad (*Uptime*) y si hay permisos de escritura en la persistencia.

5. Fases de la Simulación

La práctica se divide en dos fases cronometradas de **35 a 40 minutos** cada una:

5.1. Fase 1: La Pared de la Confusión (Silos Manuales)

Los equipos intentan desarrollar y desplegar únicamente las historias **HU1, HU2 y HU3** bajo las siguientes condiciones:

- Devs y Ops trabajan en silos físicos y no comparten pantalla ni hablan de manera síncrona.
- Despliegue manual: Ops instala dependencias y configura el cortafuegos manualmente.
- Las HUs solo se consideran “Entregadas” cuando el profesor o monitor valide que el Frontend corre en la PC de Ops consumiendo exitosamente el Backend en la otra PC.
 - *Penalización:* Cada fallo en validación suma **3 minutos** de penalización al Lead Time del equipo.

5.2. Fase 2: Adopción DevOps (Flujo Híbrido y Automatización)

Tras una sesión de retrospectiva de 10 minutos, los silos se rompen. Se aplican las siguientes reglas:

- Devs y Ops se sientan juntos y trabajan como un solo equipo integrado.
- **Primeros 15 Minutos (Migración):** El equipo debe desplegar lo que construyeron en la Fase 1 (HU1-HU3) en los servidores limpios, pero utilizando exclusivamente **scripts de automatización** (ej: scripts de Bash para abrir puertos del firewall con `ufw` o instalar el runtime, y un script `deploy.sh` para el software).
- **Siguientes 25 Minutos (Continuous Delivery):** Con el pipeline funcionando, el equipo recibe las historias **HU4 y HU5** y debe desarrollarlas y desplegarlas continuamente a través del pipeline automatizado.
- Se trabaja bajo responsabilidad compartida de metas (velocidad y estabilidad conjunta).

6. Resultados y Métricas a Reportar

Cada equipo de estudiantes debe medir y reportar de forma comparativa las siguientes métricas de rendimiento para ambas fases:

Métrica	Descripción	Resultado Fase 1	Resultado Fase 2
Lead Time	Tiempo total promedio desde que se inicia una HU hasta que se valida en producción.		
Deployment Frequency	Cantidad de despliegues exitosos realizados a producción por unidad de tiempo.		
Change Failure Rate	Porcentaje de despliegues fallidos (que requirieron rollback o corrección rápida de bugs).		
MTTR	Tiempo promedio transcurrido en resolver un fallo en producción.		

Cuadro 1: Tabla Comparativa de Métricas de Entrega y Soporte.

7. Entregables y Reporte de Laboratorio

Cada equipo presentará un reporte en el formato estándar de artículo científico (dos columnas, usando esta plantilla) que contenga:

1. **Introducción y Arquitectura:** Diagrama físico de la topología de red local del laboratorio detallando IPs, puertos y configuraciones del firewall utilizadas.

2. ****Análisis de Métricas****: Comparación cualitativa y cuantitativa de los resultados de la Tabla 1, justificando los cuellos de botella de la Fase 1.
3. ****Scripts de Automatización****: Enlace al repositorio de Git que contenga los scripts de Bash para despliegue automatizado e infraestructura como código implementados en la Fase 2.
4. ****Lecciones Aprendidas****: Relacionar la experiencia práctica de la simulación con la teoría de la Pared de la Confusión, los límites de la capacidad de Ops y los principios Lean del Value Stream Mapping (VSM).